

Compiladores

Geração de Código

Ciência da Computação | 6ª etapa

Prof. Dr. Leandro Carlos Fernandes



Geração de Código Intermediário



Geração de Código Intermediário

Corresponde a 1ª etapa do processo de síntese do modelo de *Análise e Síntese*.

Em geral, ocorre em duas fases:

- Tradução da estrutura construída na análise sintática para um código em linguagem intermediária, usualmente independente do processador.

- Tradução do código em linguagem intermediária para a linguagem simbólica do processador-alvo.

A produção efetiva do código binário é realizada por outro programa (montador).



Vantagens:

Uma mesma entrada, porém com diferentes formas de saída;

Possibilita a otimização do código intermediário para prover um código otimizado mais eficiente;

Simplifica a implementação do compilador, resolvendo gradativamente as dificuldades de passagem de código fonte para código intermediário; e

Possibilita a tradução de código intermediário para diversas máquinas.



HIR (*High Intermediate Representation*):

Usada nos primeiros estágios do compilador

Simplificação de construções gramaticais para somente o essencial para otimização/geração de código

MIR (*Medium Intermediate Representation*):

Boa base para geração de código eficiente

Pode expressar todas características de linguagens de programação de forma independente da linguagem

Representação de variáveis, temporários, registradores

LIR (*Low Intermediate Representation*):

Quase 1-1 para linguagem de máquina

Dependente da arquitetura



Há várias formas de representação de código intermediário, sendo as mais comuns:

HIR: Árvore e grafo de sintaxe.

Notações Pós-fixada e Pré-fixada.

Representações linearizadas.

MIR: Código de três endereços

Quádruplas ou triplas.

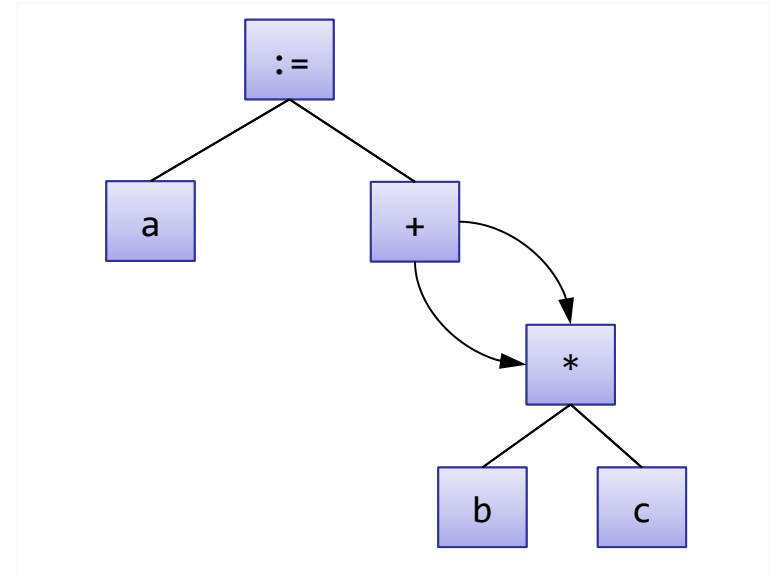
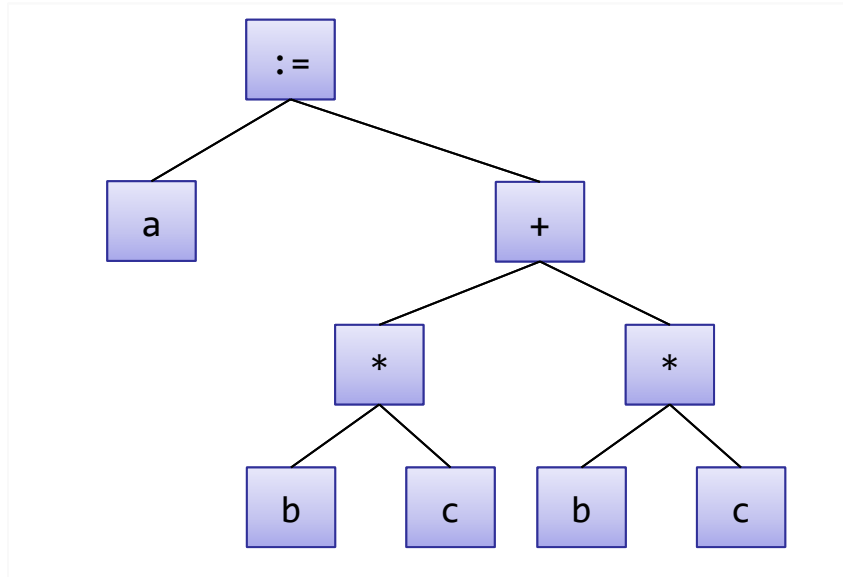
LIR: Instruções *assembler* ou *P-código*

Linguagem de montagem da arquitetura alvo.



Árvore e grafo de sintaxe

A **árvore de sintaxe** representa a estrutura hierárquica de um programa fonte enquanto o **grafo de sintaxe** inclui simplificações da árvore de sintaxe. Exemplo: $a = b * c + b * c$

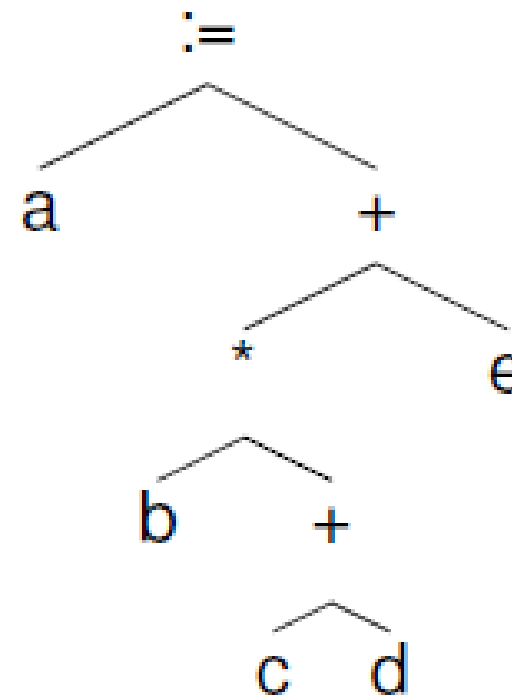


Representação Notação Pós-fixa

C++:

`a = b*(c+d)+e;`

Árvore Sintática Abstrata



Tradução:

`a b c d + * e + :=`

(varredura em pós-ordem)



Código de Três Endereços

Cada instrução referencia, no máximo, três elementos (dois operandos e um resultado).

Expressões complexas devem ser decompostas em várias expressões nesse formato

Necessita variáveis temporárias!

Formato independente de um processador específico

Fácil de traduzir para linguagem simbólica de qualquer processador.



Tipos de Instruções

Instruções de atribuição

cópia

resultado de operações
binárias

resultado de operações
unárias

Instruções de desvio

incondicional

condicional

invocação de rotinas

Operadores de endereçamento

indexado

indireto



Código de três endereços:

Instruções de atribuição

Atribuição simples

`le := ld`

Exemplo

– Código C/C++:

`x = a;`

– Tradução:

`x := a`

Código de três endereços:

Instruções de atribuição

Operação binária com atribuição

$$le := ld1 \text{ op } ld2$$

Exemplo

– Código C/C++:

$$x = a + b;$$

– Tradução:

$$x := a + b$$

Código de três endereços:

Instruções de atribuição

Operação unária com atribuição

$le := op\ ld$

Exemplo

– Código C/C++:

$x = -a;$

– Tradução:

$x := -a$

Código de três endereços: Instruções de atribuição

Expressões mais complexas são traduzidas para uma série de expressões nesse formato

- Variáveis temporárias são criadas para armazenar resultados intermediários, conforme necessário

Exemplo

- Expressão C/C++:

`a = b + c * d;`

- Tradução:

`_t1 := c * d`

`a := b + _t1`

Código de três endereços:

Instruções de desvio

Utilizados tanto para comandos de decisão ou de repetição, além de chamadas a sub-rotinas.

- Desvio incondicional
`goto L`

- Desvio condicional
`if x opr y goto L`
onde opr é um operador relacional

Código de três endereços:

Instruções de desvio

C++

```
while (i++ <= k)
    x[i] = 0;
x[0] = 0;
```

Tradução

```
_L1: if i > k goto _L2
      i := i + 1
      x[i] := 0
      goto _L1
_L2: i := i + 1
      x[0] := 0
```


Código de três endereços:

Invocando sub-rotinas

Padrão de invocação:

- Parâmetros da função, se presentes, são registrados com instrução `param`
- Sub-rotina é invocada com instrução `call`, com indicação do número de parâmetros
- Retorno, se presente, pode ser obtido a partir de atribuição do resultado de `call`

Código de três endereços: Invocando sub-rotinas

C++

```
x = f(a, g(b));
```

Tradução

```
param a  
param b  
_t1 := call g, 1  
param _t1  
x := call f, 2
```

Código de três endereços:

Modos de Endereçamento

- Modo direto

`x := a`

- Modo indexado

`x := y[i]`

`x[i] := y`

com *i* em bytes!

- Modo indireto

`x := &y`

`w := *x`

`*x := z`

Código de três endereços:

Estruturas de representação

Quádruplas

- Tabela com quatro colunas:
- Operador, primeiro argumento, segundo argumento e resultado

Triplas

- Coluna com resultado é omitida
- Referência à linha da tabela é utilizada para indicar resultados intermediários
- Operador de atribuição simples deve ser utilizado para explicitar armazenamento de resultados não temporários

Representação por Quádruplas

- C++

`a = b + c * d;`

- Quádrupla:

	Operador	Arg_1	Arg_2	Resultado
1	*	c	d	_t1
2	+	b	_t1	a

Representação por Triplas

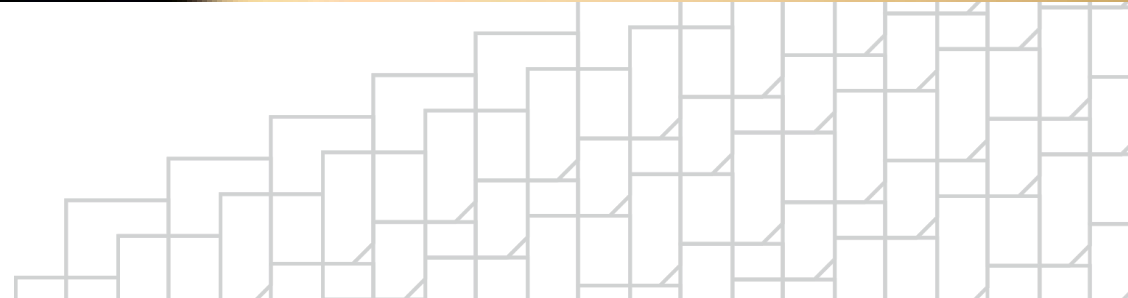
- C++

a = b + c * d;

- Tripla:

	Operador	Arg_1	Arg_2
1	*	c	d
2	+	b	(1)
3	:=	a	(2)

Otimização



Otimização de código

Código gerado automaticamente pode ser ineficiente

- Redundâncias

- Instruções desnecessárias

Ineficiência pode vir também da codificação original.

Heurísticas de otimização podem ser aplicadas no código intermediário antes da produção e conversão em linguagem simbólica.



1. Eliminação de subexpressões comuns

Operações que se repetem sem que seus argumentos sejam alterados podem ser realizadas uma única vez

Exemplo:

Válido se não houver atribuição as variáveis a ou b entre as duas instruções:

$x = a + b + c;$

...

$y = a + b + d;$



Sem otimização:

```
_t1 := a + b  
x  := _t1 + c  
...  
_t2 := a + b  
y  := _t2 + d
```

Com otimização:

```
_t1 := a + b  
x  := _t1 + c  
...  
y  := _t1 + d
```

2. Eliminação de código redundante

Instruções sem efeito podem ser eliminadas

Exemplo

Sem nenhuma atribuição a x ou a y entre as duas instruções,

x := y

x := y

...

...

x := y

z := k

z := k

a segunda instrução pode ser seguramente eliminada



3. Propagação de cópias

Variáveis que só mantêm cópia de um valor, sem outros usos, podem ser eliminadas

Exemplo:

Sem outras atribuições a y e
sem outros usos de x

x := y

...

z := x

Pode ser reduzido a:

...

z := y



4. Eliminação de desvios desnecessários

Desvio incondicional para a próxima instrução pode ser eliminado

Exemplo:

```
    a := _t2  
    goto _L6  
_L6: c := a + b
```

É equivalente a:

```
    a := _t2  
    c := a + b
```



5. Eliminação de código não-alcançável

Instruções que nunca serão executadas podem ser eliminadas.

Exemplo:

```
    goto _L3
    _t1 := x
_L4: _t2 := b + c
_L3: d  := a + _t2
```

Equivale a:

```
    goto _L3
_L4: _t2 := b + c
_L3: d  := a + _t2
```



6. Movimentação de código

Retirar do corpo de comandos iterativos (laços) cálculo de expressões invariáveis.

Exemplo:

```
while ( i < 2*max )  
    a[i] = i + max/4;
```

Como max não varia, equivale a:

```
_t1 = 2*max;  
_t2 = max/4;  
while (i < _t1)  
    a[i] = i + _t2;
```



7. Uso de propriedades algébricas

Relaciona-se a substituição de certas expressões aritméticas por formas equivalentes e de melhor desempenho.

Original	Equivalente
$X + Y$	$Y + X$
$X + 0$	X
$X - 0$	X
$X * Y$	$Y * X$
$X * 1$	X
$2 * X$	$X + X$
X^2	$X * X$



Geração de Código (Linguagem Simbólica)

	MOV	EBX, J	EBX = J	6	105
ROBERTA:	MOV	ECX, K	ECX = K	6	111
	IMUL	EAX, EAX	EAX = I * I	2	117
	IMUL	EBX, EBX	EBX = J * J	3	119
	IMUL	ECX, ECX	ECX = K * K	3	122
MARILYN:	ADD	EAX, EBX	EAX = I * I + J * J	2	125



Geração de código

Objetivo: Obter, a partir das instruções elementares usadas no código em formato intermediário, código equivalente na linguagem simbólica do processador-alvo.

Processadores diferentes podem ter formatos de instruções distintos.



Classificação pelo número de endereços na instrução:

3 end: dois operandos e o resultado.

2 end: dois operandos (o resultado sobrescreve primeiro operando).

1 end: refere-se somente ao segundo operando; o primeiro operando é implícito (registrador acumulador) e o resultado sobrescreve primeiro operando.

0 end: operandos e resultado em uma pilha, sem endereço explícitos.



Tradução para linguagem simbólica

Tradução ocorre segundo gabaritos definidos de acordo com o tipo de máquina.

Exemplo: $x := y + z$

3-end:

ADD y, z, x

2-end:

MOVE Ri, y

ADD Ri, z

MOVE x, Ri

1-end:

LOAD y

ADD z

STORE x

0-end:

PUSH y

PUSH z

ADD

POP x



Otimização em linguagem simbólica

Ao combinar sequências de instruções traduzidas pelos gabaritos, as estratégias de otimização discutidas também podem se aplicam ao código em linguagem simbólica.

- Eliminação de código redundante

- Eliminação de instruções desnecessárias



Além disso, há também a oportunidade de realizar otimizações que são específicas para um determinado processador

Otimizações dependentes de máquina

Permitem o aproveitamento de instruções pouco usuais, de uso restrito

Muitas vezes, é difícil para o compilador reconhecer a possibilidade de uso dessas instruções





Universidade Presbiteriana
Mackenzie



Faculdade de
Computação e Informática

